**Saint Michael College of Caraga**
Tel. Nos. +63 085 343-3251 / +63 085 283-3113
Fax No. +63 085 808-0892
Brgy. 4, Nasipit, Agusan del Norte, Philippines
www.smccnasipit.edu.ph

**Final Project Guide**
**Mini Social Networking Web Application**
*Web Systems and Technologies – Final Output*

## 1. Project Overview

In this final project, you will design and develop a **Mini Social Networking Web Application**, a simplified platform where users can create accounts, post updates, interact with other users, and manage their profiles.

The application must be **full stack**, using:

- **Front-end:** HTML, CSS (or Bootstrap/Tailwind), JavaScript (or JS libraries/frameworks)
- **Back-end:** PHP **following MVC architecture**
- **Database:** MySQL

Your system should simulate key features of modern social platforms in a simplified form while showcasing your mastery of full-stack development.

## 2. Project Learning Outcomes

After completing this project, you should be able to:

1. Build a dynamic full-stack web system with CRUD operations.
2. Apply **MVC principles** in organizing server-side code.
3. Design user authentication and session handling.
4. Implement user-generated content features (posts, comments, likes).
5. Perform both **client-side and server-side validation**.
6. Use MySQL for relational data storage.
7. Build a responsive and user-friendly front-end layout.
8. Integrate basic security practices (SQL injection prevention, XSS guard).

## 3. Core Features & Requirements

Your Mini Social Networking Web Application **must include the following features**:

### A. User Authentication Module

- Register (with validation)
- Login & Logout
- Password hashing (password_hash())
- Option to upload profile picture (optional but recommended)

### B. User Profile

Each user must have a **profile page** containing:

- Avatar/Profile Photo
- Full Name & Username
- Bio/About
- List of user posts

- Edit Profile (Update name, bio, photo)

**C. Posts (CRUD Module #1)**

Users must be able to:

- Create posts (text, optional image)
- View posts in a timeline/newsfeed
- Update their own posts
- Delete their own posts

**D. Comments (CRUD Module #2)**

Users must be able to:

- Comment on any post
- Edit their own comments
- Delete their own comments

You may show comments directly under each post.

**E. Likes / Reactions (Optional but Recommended)**

Implement at least:

- Like button
- Like counter

You may allow only one like per user per post.

**F. Newsfeed / Timeline**

After login, users should see:

- Posts from all users, ordered by **latest first**
- Displayed with username, timestamp, and content

Pagination or "Load More" is optional.

**G. Search & User Discovery**

Implement at least one search feature:

- Search users by name or username
  **OR**
- Search posts by keywords

**H. Responsive UI**

- Use Bootstrap, Tailwind, Bulma, or your preferred CSS framework.
- Application must be usable on laptop and mobile screens.

## 4. Technical Requirements

### A. Back-End (PHP MVC)

Follow strict **MVC separation**:

**MODELS – Data Access Layer**

- UserModel.php
- PostModel.php
- CommentModel.php
- LikeModel.php (optional)

Each model must:

- Only contain database-related logic
- Use **PDO** or **MySQLi + prepared statements**

**CONTROLLERS – Business Logic**

Controller examples:

- AuthController.php
- ProfileController.php
- PostController.php
- CommentController.php

Controllers must handle:

- Routing requests
- Calling model functions
- Passing data to views

**VIEWS – Presentation Layer**

Views should include:

- Header, footer, navigation components
- Login/Register pages
- Profile pages
- Newsfeed page
- Post forms
- Comment sections

No direct database queries inside views.

### B. Database Requirements (Minimum 4 Tables)

Required tables:

1. **users**
    - id, username, password, full_name, bio, profile_image, created_at

2. **posts**
   - id, user_id (FK), content, image, created_at
3. **comments**
   - id, post_id (FK), user_id (FK), content, created_at
4. **likes** (optional but recommended)
   - id, post_id (FK), user_id (FK)

You must submit the **ERD + SQL schema file**.

**C. Security Requirements**

- Use prepared statements (NO raw SQL with concatenation).
- Sanitize user input.
- Escape output to prevent XSS.
- Use session-based authentication.
- Limit actions to authorized users (i.e., can only edit/delete their own posts/comments).

**D. UI/UX Requirements**

- Clean and consistent interface.
- Use icons (FontAwesome, Bootstrap Icons, etc.)
- Responsive layout for mobile and desktop.
- Smooth navigation (AJAX optional).

## 5. Project Deliverables

**A. Source Code Folder (GitHub repo)**

- Complete MVC project
- Proper folder structure:

```
app/
  controllers/
  models/
  views/
public/
  index.php
  assets/
config/
  database.php
sql/
  social_app.sql
```

**B. SQL File**

- social_app.sql containing CREATE TABLE statements and sample data.

**C. Documentation (PDF or Word)**

Include the following:

1. **Cover Page**
2. **Introduction**
3. **System Objectives**
4. **System Features**
5. **Use Case Diagram**
6. **Data Flow Diagram (optional)**
7. **ERD (Entity Relationship Diagram)**
8. **Database Schema Description**
9. **Screenshots of all pages**
10. **User Guide**
11. **Team Members Contribution Report** (for group work)

**D. Final Presentation/Demo (5–10 Minutes Screen Recording)**

Present the following:

- System concept and features
- System architecture (MVC)
- Demo of Login → Newsfeed → Posting → Commenting → Liking → Profile
- Show CRUD interactions
- Explanation of security practices used
- Highlight challenges and learnings

## 6. Optional Advanced Features (Bonus +10 Points)

Students may implement:

- AJAX-based posting or dynamic comments
- Real-time notifications (via polling or Pusher API)
- Follow/Unfollow system
- Direct messaging (simple)
- Dark mode toggle
- Image cropping / file validation
- Profile cover photo

## 7. Evaluation Rubric (100 Points)

| Criteria | Points |
|---|---|
| System Functionality (Working Features) | 25 |
| User Interface & UX Design | 15 |
| MVC Architecture Implementation | 20 |
| Code Quality & Security | 15 |
| Database Design & SQL File | 10 |
| Documentation Completeness | 10 |
| Final Presentation | 5 |
| **TOTAL** | **100** |

## 9. Submission

Upload the deliverables indicated in **item no. 5** in your Google drive. And submit only the link. Make it sure that the drive is accessible to public, otherwise you will be given 0 as your final score.

Prepared by:

**MARLON JUHN M. TIMOGAN, DIT (CAR)**
Instructor